



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/26



Gruppo 17

Nome: BitByBit

Email: swe.bitbybit@gmail.com

Verbale Riunione Esterna Numero 3

Stato:	Completato
Versione:	1.0.0
Redattori:	Giovanni Visentin
Verificatori:	Ferdinando Fracasso, Marco Sanguin
Responsabile:	Dennis Parolin
Destinatari:	Prof. Tullio Vardanega Prof. Cardin Miriade

Indice

1	Informazioni generali	3
2	Ordine del Giorno	4
3	Discussioni	4
4	Decisioni	5
5	To Do	5



1. Informazioni generali

Data:	14 aprile 2026
Ora Inizio:	9:10
Durata:	30 min
Luogo:	Google Meet
Presenti:	Fracasso Ferdinando Manisi Riccardo Parolin Dennis Scaggiante Gabriele Visentin Giovanni
Docenti:	Prof. Cardin
Assenti:	Sanguin Marco



2. Ordine del Giorno

- Chiarimento sul pattern architetturale da adottare in AWS: Serverless vs Microservizi.
- Valutazione della condivisione di database tra diverse funzioni Lambda.
- Utilizzo del **Provider^G** in Flutter per la gestione dello stato e delle dipendenze.
- Dettagli sull'implementazione dell'architettura esagonale.
- Ottimizzazione della gestione dei dati statici.

3. Discussioni

- **Architettura Backend e Pattern:** È stata discussa la differenza tra architettura a microservizi e approccio serverless. Il Prof. Cardin ha chiarito che non si può definire "microservizio" un'architettura in cui più Lambda condividono lo stesso database. Attualmente il sistema è orientato verso un "serverless che tende ai microservizi", ma il consiglio principale è quello di adottare un pattern pienamente **Serverless^G**, assegnando una singola funzionalità o operazione a ciascuna funzione Lambda per evitare sovraccarichi di responsabilità.
- **Dependency Injection in Flutter:** È stato confermato che l'utilizzo del framework Provider in Flutter rappresenta la scelta architetturale più corretta. L'uso di Singleton è sconsigliato in quanto crea dipendenze implicite non evidenti dal costruttore e lega il ciclo di vita all'intera app, impedendone la ricreazione flessibile.
- **Architettura Esagonale:** Si è discusso del ruolo di **adapter^G** e interfacce. È emerso che la Business Logic non deve mai dipendere da dati o risorse esterne e deve operare unicamente utilizzando le classi di dominio. L'impiego di Inbound Adapter (come un ipotetico *SesEventAdapter*) per decodificare gli input esterni e convertirli in oggetti di dominio è un approccio validato e incoraggiato.
- **Gestione Dati Statici:** Per il recupero di dati statici che non necessitano di modifiche da parte dell'utente (come la posizione dei luoghi sicuri), è stato sconsigliato l'uso intermedio di API Gateway o Lambda verso DynamoDB. L'approccio raccomandato è archiviare tali risorse all'interno di Amazon S3, permettendo al client di accedervi direttamente per ridurre ritardi e costi operativi.



4. Decisioni

Descrizione decisione	Codice decisione
Adottare un pattern architetturale Serverless, assegnando preferibilmente una singola funzionalità (per esempio singola operazione CRUD) per ogni funzione Lambda.	VE PB 3.1
Utilizzare ufficialmente il pacchetto Provider di Flutter per la Dependency Injection e abbandonare il pattern Singleton.	VE PB 3.2
Isolare strettamente la Business Logic dai dati esterni avvalendosi di Inbound Adapter e oggetti di dominio secondo i principi dell'Architettura Esagonale.	VE PB 3.3
Migrare la memorizzazione dei dati non modificabili dall'utente su Amazon S3, eludendo passaggi superflui su Lambda e DynamoDB.	VE PB 3.4

5. To Do

Le prossime attività saranno discusse e assegnate in dettaglio durante la riunione interna di inizio Sprint. Le direttive generali emerse dall'incontro prevedono:

- Revisione e refactoring dell'architettura AWS al fine di disaccoppiare le responsabilità delle attuali Lambda condivise, orientandosi verso un pattern prettamente **Serverless^G**.
- Aggiornamento dei diagrammi delle classi e dell'architettura per riflettere le variazioni apportate alla gestione delle dipendenze e degli adapter.
- Predisposizione di bucket S3 e migrazione degli attuali dati statici, configurando opportunamente il client Flutter per il loro recupero.